

# Relatório técnico da Calculadora Científica

Guilherme Avelino Macedo - 20250026942

## 1 Introdução e Contexto

O projeto se trata de uma calculadora completa. Esta calculadora tem o objetivo de suprir as necessidades de realizar qualquer cálculo. Pois ela tem as funções de:

- Calculadora básica
- Calculadora científica
  - Função trigonométrica
  - Logaritmos ( $\log_{10}$  ou  $\ln$ )
  - Raiz quadrada
  - Potenciação
  - Fatorial
- Mostrar histórico
- Limpar histórico

### 1.1 Justificativa da escolha da calculadora

Escolhi esse projeto porque ele permite aplicar de forma prática diversos conceitos importantes da linguagem C, como estruturas de dados, manipulação de *strings*, uso de bibliotecas matemáticas e tratamento de bastantes erros. Além disso, o projeto é desafiador, que vai além das quatro operações básicas, exigindo implementação de algoritmos como o *Shunting-Yard* para análise de expressões e um sistema de registro de cálculos, o que garante uma boa complexidade e relevância prática.

## 2 Análise técnica

### 2.1 Metodologia

Assisti a todos os tutoriais de como instalar um compilador para C, seja embutido no Visual Studio Code, seja usando aplicativos externos. Tive muita dificuldade para conseguir instalar um compilador no meu computador, até que achei um tutorial me ensinando a instalar o Dev-C++, que é um software de IDE focado em linguagens C (C, C#, C++), ou seja, ele serve como um compilador e editor ao mesmo tempo. Assim, consegui finalmente começar a programar.

### 2.2 Aplicação dos conceitos da U1

#### 2.2.1 Como foram usadas as estruturas condicionais?

No projeto, as estruturas condicionais foram usadas para

- **Validação de entradas:**

Vários `if` foram usados pra verificar se a entrada do usuário era válida, tipo:

- no `menuPrincipal()`, checando se o `scanf` recebeu realmente um número;
- no `validarExpressao()`, onde cada caractere é testado (`if (!ehNumero(c) && !ehOperador(c) ...) return 0;`);
- Esses condicionais garantem que a calculadora não quebre com expressões erradas.

- **Tratamento de erros matemáticos:** Tem `if (b == 0)` em `aplicarOperacao()` pra evitar divisão por zero, e também `if (valor <= 0)` nos logaritmos e `if (valor < 0)` na raiz quadrada.

- **Controle de fluxo do programa:**

No `menuPrincipal()`, o `switch (opcao)` direciona o usuário para calculadora básica, científica, histórico, limpar ou sair.

No `calculadoraCientifica()`, tem outro `switch` que separa as funcionalidades (trigonométricas, log, raiz, potência, fatorial).

- **Operações matemáticas:**

Em `aplicarOperacao()`, o `switch` define qual operação fazer (+, -, \*, /, ^), tornando o código mais legível e organizado.

- **Repetição condicional:**

O `while (continuar == 's' || continuar == 'S')` permite que o usuário continue fazendo cálculos até decidir parar.

As estruturas condicionais foram usadas principalmente para validar entradas, tratar erros, controlar o menu interativo e definir o comportamento de cada operação matemática. Elas foram fundamentais pra dar robustez ao código e evitar que o programa aceitasse expressões inválidas ou causasse erros de execução.

## 2.3 Qual a lógica das estruturas de repetição implementadas?

- **Lazo do-while no menu principal:**

do { ... } while (opcao != 0); no menuPrincipal().  
A ideia é que o menu principal sempre apareça pelo menos uma vez, e continue aparecendo enquanto o usuário não escolher sair (opção 0).

- **Lazo while nas calculadoras:**

while (continuar == 's' || continuar == 'S').  
A lógica é dar liberdade pro usuário: ele pode repetir quantos cálculos quiser dentro da mesma sessão, e só sai quando digita outra coisa que não seja "s". É um laço de repetição controlado por sentinela (a variável continuar).

- **Lazo while na avaliação de expressões:**

while (i < len) em avaliarExpressao(). Ele percorre a expressão matemática caractere por caractere, analisando se é número, operador, parêntese, etc., até chegar ao final da *string*. A repetição aqui serve pra implementar o algoritmo *Shunting-Yard*.

- **Lazo for no cálculo do fatorial:**

Em calculadoraCientifica(), no caso do fatorial, tem um for (int i = 1; i <= n; i++) fat \*= i;. A lógica é multiplicar progressivamente os números de 1 até n.

As estruturas de repetição foram implementadas para manter o programa em funcionamento contínuo até a escolha do usuário, permitir repetição de cálculos sem reiniciar o programa, percorrer expressões matemáticas na avaliação de operações e executar cálculos que dependem de um número conhecido de iterações, como o fatorial.

## 2.4 Como os vetores foram aplicados no projeto?

- **Histórico de cálculos:**

Criei um vetor de structs (HistoricoItem historico[MAX\_HISTORICO];), onde cada posição guarda:

- a expressão digitada (char expressao[MAX\_EXPRESSAO]),
- o resultado (double resultado),
- e se o item é válido ou não (int valida).

Esse vetor funciona como uma espécie de banco de dados temporário, permitindo armazenar e exibir todos os cálculos que o usuário já fez. Quando o limite é atingido, desloca os elementos para sempre manter os mais recentes.

- **Armazenamento temporário de números e operadores:**

Na função avaliarExpressao(), implementei duas pilhas usando vetores:

- double pilhaNumeros[MAX\_EXPRESSAO] → guarda os números da expressão,
- char pilhaOperadores[MAX\_EXPRESSAO] → guarda os operadores (+, -, \*, etc.).

Essas pilhas (que são vetores controlados pelos índices topoNumeros e topoOperadores) são fundamentais para o algoritmo *Shunting-Yard* funcionar e respeitar a precedência de operadores.

- **Strings em C também são vetores:**

A própria expressão que o usuário digita (char expressao[MAX\_EXPRESSAO]) é um vetor de caracteres. Sempre que ele manipula essa *string* — seja validando, copiando ou exibindo no histórico — na verdade está trabalhando com vetores.

Os vetores foram aplicados de três formas principais: armazenamento do histórico de cálculos, implementação de pilhas para análise e cálculo das expressões, e manipulação de *strings*.

## 2.5 Organização e função das funções criadas

O código foi dividido em funções específicas, evitando que toda a lógica ficasse concentrada na função main. Cada função recebeu um nome representativo, que já indica sua finalidade e contribui para a clareza do programa. A main foi utilizada apenas como ponto central de execução, coordenando as chamadas das demais funções em vez de resolver diretamente todos os cálculos.

### 2.5.1 Funções de cálculo

Criadas para realizar operações matemáticas. Recebem parâmetros de entrada e retornam o resultado, isso elimina a repetição de código, pois o cálculo é feito em um único local.

### 2.5.2 Funções de exibição/saída

Responsáveis por mostrar mensagens e resultados ao usuário. Melhoram a organização, separando o que é processamento do que é apresentação.

### 2.5.3 Funções de controle do programa

Gerenciam o fluxo principal, como o menu de opções. Determinam qual operação será realizada, com base na escolha do usuário.

## 2.6 Estrutura de dados

### 2.6.1 Variáveis simples

Foram utilizadas para guardar valores temporários, como resultados de operações matemáticas, opções escolhidas pelo usuário e dados de entrada. Cada variável foi declarada com um tipo específico (como `int` ou `float`), de acordo com a natureza da informação que deveria ser armazenada. Esse uso permitiu o controle direto de informações pontuais necessárias para o funcionamento das funções e do fluxo do programa.

### 2.6.2 Vetores

Foram aplicados para armazenar coleções de dados relacionados, possibilitando o acesso a múltiplos valores da mesma natureza. Essa estrutura foi útil para registrar históricos de cálculos, guardar séries de resultados ou organizar entradas fornecidas de forma compacta. O uso de vetores facilitou a implementação de laços de repetição, uma vez que os índices puderam ser percorridos de maneira sistemática para processar todos os elementos armazenados.

## 3 Implementação e Reflexão

### 3.1 Dificuldades Encontradas

#### 3.1.1 Encontrar IDE com compilador

Foram realizadas diversas tentativas, com o auxílio de diferentes tutoriais, para instalar uma IDE que incluísse um compilador funcional no computador, a fim de executar as listas de exercícios e o projeto. Entretanto, não obtive sucesso. Geralmente quando eu instalava o IDE, o programa não compilava e executava.

#### 3.1.2 Complexidade do projeto

A escolha do desenvolvimento de uma calculadora, conforme descrito na seção 1.1 Justificativa da escolha da calculadora, foi motivada pela busca de um desafio que também tivesse utilidade prática. Entre as opções disponibilizadas no repositório da turma no *GitHub*, a calculadora científica achei mais interessante. Contudo, ao pesquisar sobre o funcionamento de uma calculadora, foram identificados conceitos avançados, como estruturas de pilhas e o algoritmo *Shunting-Yard*. O entendimento desses tópicos é desafiador, uma vez que demandam conhecimento mais aprofundado em estruturas de dados e lógica de programação.

### 3.2 Soluções Implementadas

#### 3.2.1 Sobre a IDE com compilador

Inicialmente, busquei tutoriais em português para configurar uma IDE com compilador, mas não obtive sucesso.

Em seguida, passei a pesquisar em inglês e encontrei um vídeo que apresentava três possíveis soluções. As duas primeiras não funcionaram, porém a terceira se mostrou eficiente: a instalação de um *fork* (uma cópia modificada e independente de um software já existente) desenvolvido pela empresa norte-americana *Embarcadero* fundada em 1993; a companhia é reconhecida pela produção de ferramentas de alta produtividade voltadas a desenvolvedores e profissionais de banco de dados. Após a instalação, consegui compilar e executar o programa corretamente, o que solucionou a dificuldade enfrentada.

#### 3.2.2 Sobre a complexidade do projeto

A solução para essa dificuldade foi encontrada por meio de pesquisas na Internet, onde há uma ampla variedade de aulas sobre diferentes conceitos, incluindo conteúdos relacionados a estruturas de pilhas e ao algoritmo *Shunting-Yard*. Assisti a diversas explicações e tentei aplicar sozinho o que estava aprendendo, mas encontrei dificuldade em compreender alguns conceitos e acabei ficando travado em determinadas partes. Diante disso, recorri ao uso de *LLMs* (*Large Language Models*) e também a artigos e fóruns de programação disponíveis gratuitamente na Internet. Esses recursos foram fundamentais tanto para compreender melhor a parte teórica do tema quanto para auxiliar em problemas de sintaxe e lógica de programação que não estava conseguindo resolver de forma independente.

#### 3.2.3 Organização do Código

Optei por organizar o código em funções específicas para cada necessidade, para evitar que toda a lógica ficasse concentrada na função `main`, conforme ordenado pelo professor “*funções (pelo menos 3 funções além da main)*”. O `main` foi utilizado apenas como ponto central de execução, responsável por coordenar as chamadas das demais funções, enquanto as operações mais detalhadas foram distribuídas em blocos bem definidos. As funções de cálculo foram criadas para executar operações matemáticas de forma organizada, eliminando repetições no código. As funções de exibição, por sua vez, foram destinadas a apresentar resultados e mensagens ao usuário, permitindo separar a lógica interna da interface de saída. Por fim, as funções de controle assumiram a responsabilidade de gerenciar o fluxo principal do programa, como a navegação pelo menu e a determinação das operações a serem realizadas.

#### 3.2.4 Conclusão (ainda a concluir)

O desenvolvimento da calculadora científica possibilitou ampliar meus conhecimentos em programação em C, desde o uso de estruturas condicionais, laços de repetição e vetores até a implementação de conceitos mais complexos, como pilhas e o algoritmo *Shunting-Yard*. Durante o processo, enfrentei dificuldades tanto técnicas, como a configuração de uma IDE adequada, quanto conceituais, especialmente na compreensão de algoritmos avançados,

mas consegui superá-las por meio de pesquisa em tutoriais, artigos, fóruns e uso de *LLMs*. Além do aprendizado técnico, percebi a importância de organizar o código em funções claras e independentes. Como melhoria, reconheço a necessidade de planejar melhor o tempo de realizar as atividades de qualquer disciplina, evitando realizar a entrega tão próxima do prazo final.

## 4 Perguntas Orientadoras

### 4.1 Quais conceitos da Unidade 1 foram aplicados e onde?

As variáveis simples foram utilizadas para armazenar dados temporários, como os valores de entrada, resultados intermediários e opções escolhidas pelo usuário, garantindo o controle das informações necessárias para a execução das funções. Já os vetores foram empregados para gerenciar coleções de dados, como o histórico de cálculos (`HistoricoItem historico[MAX_HISTORICO]`) e as pilhas de números e operadores (`pilhaNumeros` e `pilhaOperadores`) usadas na implementação do algoritmo *Shunting-Yard*.

As estruturas condicionais (`if`, `switch`) foram aplicadas para validar entradas, tratar erros matemáticos (como divisão por zero e logaritmos de números não positivos), controlar o fluxo do programa e definir o comportamento das operações matemáticas. Quanto às estruturas de repetição (`while`, `for`, `do-while`), elas foram usadas para permitir que o usuário repita cálculos, percorrer expressões matemáticas caractere por caractere durante a avaliação e realizar operações que exigem iterações conhecidas, como o cálculo do fatorial.

### 4.2 Como a organização em funções facilita a manutenção do código?

A organização do código em funções facilita significativamente a manutenção, porque cada função é responsável

por uma tarefa específica e bem definida. Isso permite que erros ou melhorias sejam tratados em um único ponto, sem a necessidade de modificar várias partes do programa. Além disso, funções com nomes claros tornam o código mais legível e compreensível, permitindo que qualquer pessoa que leia ou retome o projeto consiga entender rapidamente como funciona o *software*.

### 4.3 Como os vetores foram utilizados para resolver o problema proposto?

Os vetores foram fundamentais para gerenciar e organizar dados de forma eficiente. Eles foram utilizados principalmente para armazenar o histórico de cálculos (`HistoricoItem historico[MAX_HISTORICO]`), permitindo registrar e acessar todas as expressões e resultados previamente calculados. Além disso, foram aplicados como pilhas na função de avaliar expressões (`pilhaNumeros` e `pilhaOperadores`), essenciais para a implementação do algoritmo *Shunting-Yard*, garantindo que a precedência de operadores fosse respeitada. Por fim, as *strings* em C, que são vetores de caracteres, foram manipuladas para validar, copiar e exibir expressões.

### 4.4 Quais foram os principais desafios na implementação das estruturas de repetição?

Os principais desafios foram controlar corretamente o fluxo dos laços para evitar *loops* infinitos e garantir que o usuário pudesse repetir cálculos ou sair do programa. Na avaliação de expressões, foi necessário percorrer a *string* e gerenciar as pilhas de números e operadores respeitando a precedência das operações. Já no cálculo do fatorial, o cuidado foi evitar estouro de valores e garantir precisão.